PHP Object-Oriented Programming (OOP)

Contents

1 PHP Object-Oriented Programming (OOP)

Object-oriented programming (OOP) is a programming paradigm that organizes software using objects. Objects are self-contained units of data that encapsulate both data and the operations that can be performed on it. OOP provides several advantages over procedural programming, including:

- **Modular design:** OOP promotes modular design by dividing programs into independent objects that can be easily reused and composed.
- **Encapsulation:** OOP facilitates data encapsulation, which means that data is protected from direct access and can only be modified through the object's methods. This promotes data integrity and prevents accidental data corruption.
- **Inheritance:** OOP enables code reuse by allowing new classes to inherit properties and methods from existing classes. This reduces code duplication and promotes code maintainability.
- **Polymorphism:** OOP supports polymorphism, which allows objects of different classes to respond to the same method call in different ways. This promotes flexibility and adaptability in code.

PHP is a scripting language that supports OOP concepts. It provides a rich set of features for defining classes, creating objects, and managing object interactions. This guide provides a comprehensive overview of OOP in PHP, covering essential concepts, advanced techniques, and best practices.

1.1 Class

A class is the blueprint for creating objects. It defines the structure of an object, including its properties (data) and methods (behaviors).

Object Instantiation

An object is an instance of a class. It represents a specific instance of the data and behavior defined in the class. To create an object, use the `new` keyword followed by the class name.

```php
<?=
  class Student {
      public $name;
      public $age;

      public function __construct($name, $age) {
          $this->name = $name;
          $this->age = $age;

          echo "Creating a new student: {$this->name} ({$this->age})";
      }
  }

  $student1 = new Student('John Doe', 25);
  $student2 = new Student('Jane Doe', 22);
```

Accessing Object Members

Object members (properties and methods) can be accessed using the dot notation. For properties, use the accessor `$this->` to access them within the object's methods. For methods, use the object name followed by the method name and parentheses to call the method.

```php
<?=
$student1->name = 'John Smith'; // Set the name property
echo $student1->name; // Get the name property

$student2->sayHello(); // Call the sayHello() method
```

Initial Properties Values

Properties can be assigned initial values when declaring the class. These values are assigned to the object's properties when the object is instantiated.

```php
<?=
class Student {
    public $name = 'John Doe';
    public $age = 25;
}

$student1 = new Student();
echo $student1->name; // Output: John Doe
```

```php
echo $student1->age; // Output: 25
```

Constructor

The constructor is a special method that is called when an object is created. It is typically used to initialize the object's properties. The constructor is defined using the `__construct()` method.

```php
<?=
class Student {
    public function __construct($name, $age) {
        $this->name = $name;
        $this->age = $age;
    }
}

$student1 = new Student('John Doe', 25); // Initialize the object's properties
```

Destructor

The destructor is a special method that is called when an object is destroyed. It is typically used to perform cleanup tasks, such as closing database connections or releasing resources.

```php
<?=
class Student {
    public function __destruct() {
        echo "Destroying student object";
    }
}

$student1 = new Student('John Doe', 25);
$student2 = new Student('Jane Doe', 22);

unset($student1); // Destroy the student1 object
 unset($student2); // Destroy the student2 object
```

Case Sensitivity

PHP class names and object names are case-sensitive. This means that `Student` and `student` represent different classes or objects.

Object Comparison

Objects can be compared using the == and != operators. These operators compare the references of objects, not their values. To compare the values of objects, use the appropriate accessor methods or properties. For example, to compare the name property of two Student objects, you would use the following code:

```php
<?=
$student1 = new Student('John Doe', 25);
$student2 = new Student('Jane Doe', 22);
```

```php
if ($student1->getName() == $student2->getName()) {
    echo "The students have the same name.";
} else {
    echo "The students do not have the same name.";
}
```

If you want to compare objects based on their type, you can use the `instanceof` operator. For example, to check if an object is an instance of the Student class, you would use the following code:

```php
<?=
  $person = new Student('John Doe', 25);

  if ($person instanceof Student) {
      echo "The person is a student.";
  } else {
      echo "The person is not a student.";
  }
```

1.2 Anonymous Class

An anonymous class, also known as a lambda or closure, is a class without a name. It is defined using the `new` keyword followed by a block of code. Anonymous classes are often used for creating temporary objects or for passing objects as function arguments.

```php
$anonymousStudent = new class ('John Doe', 25) {
    public function sayHello() {
        echo "Hello, my name is {$this->name}.";
    }
};

$anonymousStudent->sayHello(); // Output: Hello, my name is John Doe.
```

1.3 Inheritance

Inheritance is a key concept in OOP that allows new classes to inherit properties and methods from existing classes. This promotes code reuse and reduces code duplication.

Overriding Members

Classes that inherit from other classes can override existing properties and methods. This allows for the customization of behavior without modifying the original class.

```php
class Animal {
    public function makeSound() {
        echo "The animal makes a sound.";
    }
}

class Dog extends Animal {
```

```
    public function makeSound() {
        echo "Woof!";
    }
}

$dog = new Dog();
$dog->makeSound(); // Output: Woof!
```

Final Keyword

The `final` keyword can be used to prevent a class or method from being overridden. This ensures that the original implementation is not changed by subclasses.

```
class Animal {
    final public function makeSound() {
        echo "The animal makes a sound.";
    }
}

class Dog extends Animal {
    // Cannot override the final makeSound() method
    public function makeSound() {
        echo "Woof!";
    }
}
```

instanceof Operator

The `instanceof` operator is used to check whether an object is an instance of a particular class.

```
$dog = new Dog();
if ($dog instanceof Animal) {
    echo "The dog is an instance of the Animal class.";
} else {
    echo "The dog is not an instance of the Animal class.";
}
```

1.4 Access Levels

Access levels determine the accessibility of properties and methods within a class. There are three access levels in PHP:

- **Private:** Properties and methods declared as `private` can only be accessed within the class itself.
- **Protected:** Properties and methods declared as `protected` can be accessed within the class and within subclasses of the class.
- **Public:** Properties and methods declared as `public` can be accessed anywhere in the program.

var Keyword

The `var` keyword is deprecated in PHP and is not recommended for use. It is better to use explicit access modifiers for clear and concise code.

Object Access

Access modifiers determine the access level of class members when accessing them from outside the class. For example, a `public` property can be accessed directly using the object name, while a `protected` property needs to be accessed using the `->` operator and a `public` method.

```
class Animal {
    public $name;

    protected $age;

    public function getAge() {
        return $this->age;
    }
}

$dog = new Animal();
$dog->name = 'Max'; // Access public property directly
$dog->getAge(); // Access protected property through method
```

Access Level Guideline

In general, it is recommended to use `private` for properties that should not be accessed outside the class, `protected` for properties that should only be accessible within the class and subclasses, and `public` for properties and methods that need to be accessed from anywhere in the program.

Accessors

Accessors are methods that provide controlled access to properties. They are used to set and get property values while also performing validation or other operations.

Sure, here is the completed code for the `Animal` class:

```
class Animal {
    private $name;
    private $age;

    public function setName($name) {
        if (is_string($name)) {
            $this->name = $name;
        } else {
            throw new Exception('Invalid name');
        }
    }

    public function getName() {
        return $this->name;
```

```
    }

    public function setAge($age) {
        if (is_int($age) && $age >= 0) {
            $this->age = $age;
        } else {
            throw new Exception('Invalid age');
        }
    }

    public function getAge() {
        return $this->age;
    }
}
```

The code includes additional validation for the `setAge()` method to ensure that the age is an integer and a non-negative value. This helps to maintain data integrity and prevent errors caused by invalid age values.

1.5 Static

Static members are shared amongst all instances of a class and are not bound to specific objects. They are declared using the `static` keyword.

Referencings Static Members

Static members can be accessed using the class name instead of an object.

```
class Animal {
    public static $species = 'Mammal';

    public function __construct() {
        static::$species = 'Dog'; // Modify static property
    }
}

$dog = new Animal();
echo Animal::$species; // Output: Dog
```

Static Variables

Static variables can be used to store shared data that is independent of object instances.

```
class Animal {
    private static $count = 0;

    public function __construct() {
        static::$count++;
    }
```

```
}

$dog1 = new Animal();
$dog2 = new Animal();

echo Animal::$count; // Output: 2
```

Late Static Binding

Late static binding allows method calls to be resolved at runtime, even when the method is not yet defined. This is useful for dynamic class loading and inheritance hierarchies.

```
class Animal {
    public static function sayHello() {
        return 'The animal says hello.';
    }

    public static function getGreetingMethod() {
        return self::sayHello; // Late static binding
    }
}

class Dog extends Animal {
    public static function sayHello() {
        return 'Woof!';
    }
}

$dog = new Dog();
echo Animal::getGreetingMethod(); // Outputs: Woof!
```

1.6 Interface

An interface defines a set of abstract methods that must be implemented by classes that implement the interface. Interfaces are used to enforce contracts and promote code reusability.

Interface Signatures

Interfaces specify the method names, argument lists, and return types. They do not contain implementation details.

```
interface AnimalInterface {
    public function makeSound();
}

class Dog implements AnimalInterface {
    public function makeSound() {
        echo "Woof!";
    }
```

```
}

class Cat implements AnimalInterface {
    public function makeSound() {
        echo "Meow!";
    }
}
```

Interface Example

Interfaces can be used to define common behavior for related classes.

```
interface ShapeInterface {
    public function calculateArea();
}

class Square implements ShapeInterface {
    private $sideLength;

    public function __construct($sideLength) {
        $this->sideLength = $sideLength;
    }

    public function calculateArea() {
        return $this->sideLength * $this->sideLength;
    }
}

class Circle implements ShapeInterface {
    private $radius;

    public function __construct($radius) {
        $this->radius = $radius;
    }

    public function calculateArea() {
        return pi() * pow($this->radius, 2);
    }
}
```

Interface Usage

Interfaces can be used to create objects that can interact with each other based on their common behavior.

```
$square = new Square(5);
echo $square->calculateArea(); // Output: 25

$circle = new Circle(3);
```

```
echo $circle->calculateArea(); // Output: 28.26
```

Interface Guideline

Use interfaces to define common behavior for related classes and promote code reusability.

1.7 Abstract

An abstract class is a class that contains at least one abstract method. Abstract methods do not have implementations and must be implemented by subclasses. Abstract classes are used to define partial implementations and enforce contracts.

Abstract Methods

Abstract methods are declared using the `abstract` keyword.

```
abstract class AnimalAbstract {
    abstract public function makeSound();
}

class Dog extends AnimalAbstract {
    public function makeSound() {
        echo "Woof!";
    }
}

class Cat extends AnimalAbstract {
    public function makeSound() {
        echo "Meow!";
    }
}
```

Abstract classes and interfaces are both powerful tools in object-oriented programming (OOP) that can be used to define and structure classes. While they share some similarities, they also have distinct differences.

Similarities:

- **Encapsulation:** Both abstract classes and interfaces promote encapsulation by encouraging the use of protected and private access modifiers for properties and methods. This helps to protect data from unauthorized access and maintain data integrity.
- **Code Reusability:** Both abstract classes and interfaces can be used to promote code reusability by defining common behavior or structure that can be inherited by subclasses. This reduces redundancy and improves maintainability of code.
- **Type Safety:** Both abstract classes and interfaces can help to improve type safety by specifying the types of data expected and returned by methods. This helps to catch potential errors and maintain code consistency.

Differences:

- **Purpose:** Abstract classes are more focused on defining the overall structure or blueprint of a class, including its attributes and behavior. They can contain both abstract and concrete methods. Interfaces, on the other hand, are specifically designed to define a set of behaviors that must be implemented by classes that implement the interface. They only contain abstract methods.
- **Implementation:** Abstract classes can contain concrete methods, which provide default implementations for methods that are likely to be similar across subclasses. Interfaces, however, do not contain any implementation details, leaving that responsibility entirely to the implementing classes.
- **Relationship:** Abstract classes can be related to each other through inheritance, forming a hierarchy of classes. Interfaces, on the other hand, are more independent of each other, and a class can implement multiple interfaces without being related to them through inheritance.

When to Use:

- **Abstract Classes:** Use abstract classes when you want to define a common structure or blueprint for related classes, including both common properties and behavior. You can also use abstract classes to enforce encapsulation and promote type safety.
- **Interfaces:** Use interfaces when you want to define a set of behaviors that must be implemented by classes. Interfaces are particularly useful for decoupling classes and promoting polymorphism.

In summary, abstract classes and interfaces are both valuable tools in OOP, each with its unique strengths and applications. Abstract classes are more focused on defining class structure, while interfaces are specifically designed to define behaviors. The choice between the two depends on the specific requirements and design of your application.

1.8 Traits

Traits are a feature introduced in PHP 5.4 that allows the sharing of functionality between unrelated classes. Traits can contain methods, properties, and constants.

Inheritance & Traits

Traits can be inherited from multiple classes, and their methods can be overridden or extended.

```
trait Logger {
    public function log($message) {
        echo "Logging: $message";
    }
}

class Animal {
    use Logger;

    public function makeSound() {
        echo "The animal makes a sound.";
```

```
    }
}

class Dog extends Animal {
    public function makeSound() {
        echo "Woof!";
        $this->log('Dog made a sound.');
    }
}

$dog = new Dog();
$dog->makeSound(); // Logs "Logging: Dog made a sound."
```

Trait Guidelines

Use traits to share common functionality between unrelated classes without the need for inheritance.

1.9 Overloading

Overloading is a technique in OOP that allows multiple methods with the same name to exist in a class. Overloading is based on the method's parameter types and order.

Property Overloading

Property overloading is not supported in PHP. However, it is possible to achieve similar functionality using setters and getters.

Method Overloading

Method overloading is supported in PHP using the same method name and different parameter lists.

```
class Number {
    public function add($x, $y) {
        return $x + $y;
    }

    public function add($x, $y, $z) {
        return $x + $y + $z;
    }
}

$number = new Number();
echo $number->add(10, 20); // Output: 30
echo $number->add(10, 20, 30); // Output: 60
```

isset & unset Overloading

`isset()` and `unset()` can be overloaded to check and unset properties based on their types.

```
class Number {
    public function __isset($name) {
        if (is_int($name)) {
            return $this->$name !== null;
        } else {
            return false;
        }
    }

    public function __unset($name) {
        if (is_int($name)) {
            $this->$name = null;
        }
    }
}

$number = new Number();
$number->x = 10;

if (isset($number->x)) {
    echo "x is set";
} else {
    echo "x is not set";
} // Output: x is set

unset($number->x);

if (isset($number->x)) {
    echo "x is set";
} else {
    echo "x is not set";
} // Output: x is not set
```

Overall, OOP is a powerful and versatile programming paradigm that can significantly improve the maintainability, reusability, and flexibility of PHP applications.